



Assignment Cover Page

Assignment Title:	Mid Assignment		
Assignment No:	01	Date of Submission:	2 November 2021
Course Title:	Compiler Design		
Course Code:	Click here to enter text.	Section:	G
Semester:	Fall	2021-22	Course Teacher: Md. Masum Billah

Declaration and Statement of Authorship:

1. I/we hold a copy of this Assignment/Case-Study, which can be produced if the original is lost/damaged.
2. This Assignment/Case-Study is my/our original work and no part of it has been copied from any other student's work or from any other source except where due acknowledgement is made.
3. No part of this Assignment/Case-Study has been written for me/us by any other person except where such collaboration has been authorized by the concerned teacher and is clearly acknowledged in the assignment.
4. I/we have not previously submitted or currently submitting this work for any other course/unit.
5. This work may be reproduced, communicated, compared and archived for the purpose of detecting plagiarism.
6. I/we give permission for a copy of my/our marked work to be retained by the Faculty for review and comparison, including review by external examiners.
7. I/we understand that Plagiarism is the presentation of the work, idea or creation of another person as though it is your own. It is a form of cheating and is a very serious academic offence that may lead to expulsion from the University. Plagiarized material can be drawn from, and presented in, written, graphic and visual form, including electronic data, and oral presentations. Plagiarism occurs when the origin of them material used is not appropriately cited.
8. I/we also understand that enabling plagiarism is the act of assisting or allowing another person to plagiarize or to copy my/our work.

* Student(s) must complete all details except the faculty use part.

** Please submit all assignments to your course teacher or the office of the concerned teacher.

Group Name/No.:

No	Name	ID	Program	Signature
1	Saidul Mursalin Khan	19-41766-3	BSc [CSE]	
2			Choose an item.	
3			Choose an item.	
4			Choose an item.	
5			Choose an item.	
6			Choose an item.	
7			Choose an item.	
8			Choose an item.	
9			Choose an item.	
10			Choose an item.	

Faculty use only

FACULTY COMMENTS	Marks Obtained	
	Total Marks	

Ans: to: the: Q: NO:-1

The given ambiguous grammar,

$$R \rightarrow R + R / R . R / R^* / a / b$$

We will implement the precedence and associativity constraints in order to convert the grammar into unambiguous grammar.

The grammar contains three operators and two operands. The operators are $+$, $.$ and $*$. The operands are a and b .

An ambiguous grammar can be converted into an unambiguous

grammar by implementing

precedence constraints and

Associativity Constraints. These constraints follows some rules.

As precedence constraints rules the priority order is,

$$(a, b) > * > \cdot > +$$

As associativity constraints rules, ~~the~~ $\cdot$ and $+$ operator is left associative.

Using the precedence and associativity rules we get the unambiguous grammar,

$$R \rightarrow R + T \mid T$$

$$T \rightarrow T \cdot F \mid F$$

$$F \rightarrow R^* \mid a \mid b$$

Ans: to: the: Q: No: -2

The given ambiguous grammar,

$\text{bexp} \rightarrow \text{bexp} \text{ or } \text{bexp} / \text{bexp} \text{ and } \text{bexp} /$

not $\text{bexp} / \text{T} / \text{F}$

Here, bexp represents Boolean expression,

T represents True and F represents False.

We implement the precedence and associativity constraints to convert ambiguous grammar into unambiguous grammar.

The grammar consists of three operators and two operands. The operators are or, and and not.

The operands are T and F.

As the precedence and associativity rules, the priority order is,

$(T, F) > \text{not} > \text{and} > \text{or}$

Here, and, operator and or operator is left associative.

Using the precedence and associativity

rules, we get the unambiguous grammar,

$\text{bexp} \rightarrow \text{bexp} \text{ or } A/A$

$A \rightarrow A \text{ and } B/B$

$B \rightarrow \text{not bexp} / T/F$

Ans: to the Q: NO: - 3

Syntax Directed Definition:

Syntax directed definition is one kind of abstraction of context free grammar. In syntax ~~defin~~ directed definitions, the grammar symbols have an associated set of attributes. In order to compute the values of attributes, productions should be associated. Such formalism

generates annotated parse trees

where each node of the tree is a record with a field for each attribute.

Each grammar symbol is associated with two finite sets of values.

The synthesized attributes and the inherited attributes.

Ans to the Q: NO:- 4

An syntax directed translation is a context free grammar with program fragments embedded within production bodies. Those program fragments are called semantic actions. They can appear at any position within production body. Any syntax directed translation can be implemented by first

building a parse tree and then performing the actions in a left-to-right depth first order. Typically syntax directed translations are implemented during parsing without building a parse tree.

Example:

Production	Semantic Rules
$E \rightarrow E + E$	$\{E.VAL := E.VAL + E.VAL\}$
$E \rightarrow (E)$	$\{E.VAL := E.VAL\}$
$I \rightarrow I \text{ digit}$	$\{I.VAL := 10 * I.VAL + \text{LEXVAL}\}$

Ans: to: the: Q: No: -5

A syntax directed translation is a technique where the ~~language's~~ structure of the language processor (a compiler) is based on the structure of the language's abstract syntax. There might be one procedure in the translator corresponding to each category in the abstract syntax.

That procedure is responsible for processing constructs of that category. Each procedure would call others corresponding to the construct's subconstituents and then combine their results to give the overall result for that construct.

Example:

Production	Semantic Rules
$E \rightarrow T$	$E.val := T.val$
$T \rightarrow T * F$	$T.val := T.val + F.val$
$F \rightarrow num$	$F.val := num.lexval$